

Systems Verification

Exercise Sheet 04 - Solutions

Exercise 1: Modelling the Wolf, Sheep and Cabbage problem

The aim of the exercise is to model a well-known logical problem with *Promela* language, and see if *Spin* can solve it for us. The problem is the following:

- A farmer wants to cross a river (starting from the left side) with a wolf, a sheep and a cabbage.
- However, he can take only one item at a time on his boat. Besides, he cannot leave the sheep alone with the cabbage, nor the wolf alone with the sheep on either shore.

Your task is to model this problem in *Promela*, so that *Spin* can find a correct path to the solution. The correct final situation should be represented by an assertion violation (because *Spin* will do its best to find such a violation, and the trail created will be the shortest path to the solution).

Hints:

- Use *mtypes* to specify the various actors (wolf, sheep and cabbage).
- An execution should non-deterministically choose between any possible action (i.e. choose between carrying a wolf, a sheep or a cabbage if they are present, or nothing).
- The final solution should be modelled as an assertion violation.
- Bad situations (e.g. wolf and sheep alone) should block the execution of the branch, so that *Spin* does not go any further in the simulation of that branch.
- You can use arrays to represent the items present on each shore.
- You can use channels to transfer the items from one shore to the other.

Run your model in *Spin*. It should detect an error (meaning a solution).

Switch to simulation mode to navigate through the correct solution and write it down (in English, not *Promela*).

Remark: verification is done with the following options:

Safety	Storage Mode	Search Mode
☒ safety ☐ + invalid endstates (deadlock) ☒ + assertion violations ☐ + xr/xs assertions	☒ exhaustive ☐ + minimized automata (slow) ☐ + collapse compression ☐ hash-compact ☐ bitstate/supertrace	☐ depth-first search ☒ + partial order reduction ☐ + bounded context switching with bound: 0
Liveness ☐ non-progress cycles ☐ acceptance cycles ☐ enforce weak fairness constraint	Never Claims ☒ do not use a never claim or ltl property ☐ use claim claim name (opt):	☐ + iterative search for short trail ☐ breadth-first search ☒ + partial order reduction ☒ report unreachable code
Run Stop		Save Result in: pan.out

Solution:

```
mtype = {wolf, sheep, cabbage, nothing};

mtype leftShore[3];
mtype rightShore[3];
chan boat = [1] of {mtype}
bool onLeftShore = 1

active proctype leftShoreProc(){
    leftShore[0] = wolf;
    leftShore[1] = sheep;
    leftShore[2] = cabbage;
    boat!nothing;

    startProc0:
    onLeftShore;

    if
        /* get content of boat */
        :: boat?wolf; leftShore[0]=wolf;
        :: boat?sheep; leftShore[1]=sheep;
        :: boat?cabbage; leftShore[2]=cabbage;
        :: boat?nothing;
    fi;

    if
        /* send item on boat */
        :: leftShore[0] == wolf; boat!wolf; leftShore[0] = 0;
        :: leftShore[1] == sheep; boat!sheep; leftShore[1] = 0;
        :: leftShore[2] == cabbage; boat!cabbage; leftShore[2] = 0;
        :: boat!nothing;
    fi;
    !(leftShore[0] == wolf && leftShore[1] == sheep);
    !(leftShore[1] == sheep && leftShore[2] == cabbage);

    onLeftShore = 0;
    goto startProc0;
}

active proctype rightShoreProc(){
    startProc1:
    !onLeftShore;

    if
        /* get content of boat */
        :: boat?wolf; rightShore[0]=wolf;
        :: boat?sheep; rightShore[1]=sheep;
        :: boat?cabbage; rightShore[2]=cabbage;
        :: boat?nothing;
    fi;

    assert(!(rightShore[0]==wolf && rightShore[1]==sheep && rightShore[2]==cabbage));
```

```
if
  /* send item on boat */
  :: rightShore[0] == wolf; boat!wolf; rightShore[0] = 0;
  :: rightShore[1] == sheep; boat!sheep; rightShore[1] = 0;
  :: rightShore[2] == cabbage; boat!cabbage; rightShore[2] = 0;
  :: boat!nothing;
fi;

!(rightShore[0] == wolf && rightShore[1] == sheep);
!(rightShore[1] == sheep && rightShore[2] == cabbage);

onLeftShore = 1;
goto startProc1;
}
```

Solution to the wolf, sheep and cabbage problem (in English): The boat carries the sheep from the left shore to the right shore. The empty boat goes back to the left shore and brings the wolf to the other side. Then, it brings the sheep back on the left side and carries the cabbage to the right. Finally, it goes back to the left shore and brings the sheep to the right side.